

Power and Timing Optimization Using Multibit Flip-Flop

Sheng-Wei Yang, Tzu-Hsuan Chen, Jhih-Wei Hsu, Ting Wei Li, Cindy Shen

Synopsys, Inc.

0. Revision History

2024-02-14 - Second Revision. Added the definition of 'displacement' and QpinDelay with more pictures. Also clarify the goal.

2024-02-01 - First Major Revision

1. Introduction

In modern technology nodes, the power and area minimization has become one of the most studied topics in semiconductor industry. One commonly applied concept is to replace single-bit flip-flops with multibit flip-flops. By using one multibit flip-flop to replace multiple single-bit flip-flops, more area can be freed up in the design as one multibit flip-flop takes less area to place than the single-bit flip-flops it replaces. Furthermore, before changing single-bit flip-flops to a multibit-flip-flop, each single-bit flip-flop has its own power, ground, and clock pin connections. Replacing them with one multibit-flip-flop can also efficiently reduce power, ground, and clock net routing complexity. Therefore, in modern technology nodes, this technique has been widely applied and referred to as “multibit flip-flop banking”.

However, with the advance in technology nodes, timing, power, and area optimization has become a much more convoluted problem. For some timing critical nets, the original idea of multibit-flip-flop banking attempts would possibly result in worsening the timing, which hampers the overall optimization objective. Therefore, sometimes we have to divide a multibit flip-flop into several single-bit flip-flops to further optimize timing critical nets. This technique is oftentimes referred to as “multibit flip-flop debanking”.

In this contest problem we will simulate the multibit flip-flop banking and debanking decisions in some virtual designs as testcases so that contestants would need to take timing, power, and area objectives together to find the best possible optimization solutions for each testcase.

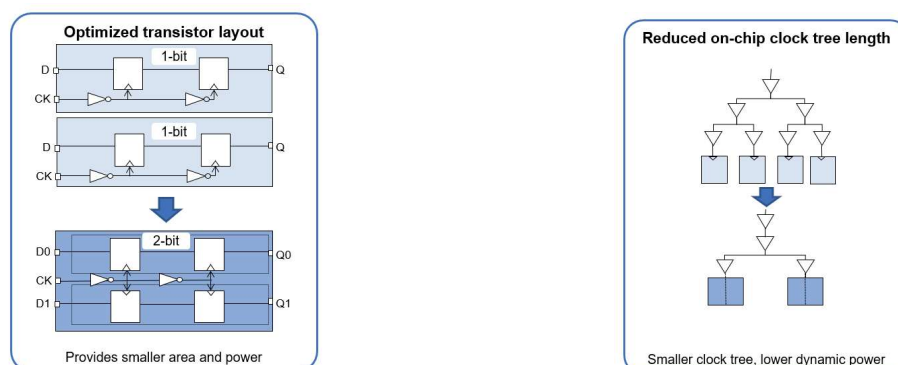


Figure 1(a): A simple transistor layout of two single-bit flip-flops and one multibit flip-flop. (left)

Figure 1(b): A diagram illustrating how dynamic power is reduced due to the reduction of on-chip clock tree length. (right)

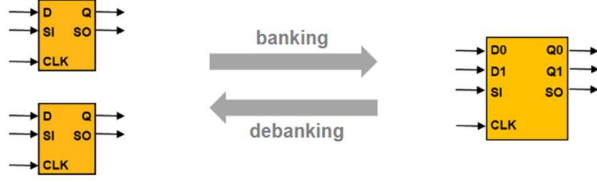


Figure 2: A simplified diagram of how two single-bit flip-flops can be banked into one two-bit flip-flop, and debanked vice versa. In this diagram, the upper left D pin of the single-bit flip-flop is mapped to the D0 pin of the two-bit flip-flop; the lower left D pin of the single-bit flip-flop is mapped to the D1 pin of the two-bit flip-flop.

2. Contest Objective

The contestants need to develop a banking & debanking algorithm that can take any given testcase and produce a placement result that satisfies cell density constraints and with the resultant timing, power, and area optimized. No cell overlapping is allowed in the placement result. A cost metric will be given for each testcase to identify the weight for each timing, power, and area calculation. The cost metrics of calculating timing, power, and area for this contest is as follows:

$$\sum_{\forall i \in FF} (\alpha \cdot TNS(i) + \beta \cdot Power(i) + \gamma \cdot Area(i)) + \delta \cdot D$$

Where i stands for every flip-flop instance in the design. $TNS(i)$ stands for the total negative slack of the flip-flop. $Power(i)$ stands for the power consumption of the flip-flop. $Area(i)$ stands for the area cost of the flip-flop. The design is divided into several bins, and we will define a utilization rate threshold for each bin in the design. D stands for the number of bins that violates the utilization rate threshold. α , β , γ , and δ are weight for each cost. The weighted summation of the four cost metrics above represents quality of result used in this contest.

Figure 3 shows an example of the density constraint of a placement. A whole placement region is divided into several uniform bins. For each bin, we set up a density threshold representing the upper bound of the cell area allowed to be placed within the bin. Should the total cell area exceeds the threshold, we consider this bin violates the density constraints, hence accrue the penalty score by density weight δ . In the contest, [the same threshold is applied to every uniform bin, which is defined in the BinMaxUtil of the input file.](#)



Figure 3: A representation of cell density constraint. The whole placement region is divided into several uniform bins, and a threshold is applied to every bin. Should the total cell area within the bin exceeds the threshold, density constraint penalty score is accrued.

3. Problem Formulation and Input/Output Format

Given:

1. Weight factors for cost metrics
2. Die size and input output ports
3. Cell library and flip-flops library information
4. A cell placement result with flip-flops cells
5. Max placement utilization ratio
6. Netlist
7. Timing slack and delay information
8. Power information

Output:

1. A cell placement solution
2. Netlist connectivity

The resultant cell placement solution must have no cell overlaps while satisfying max placement utilization ratio. The max placement utilization ratio is a cell density constraint implemented by dividing the design into several placement bins, and each bin would have a maximum ratio defining the number of cells allowed in each bin.

3.1 Format of Input Data

Weight factors of the cost metrics: α , β , γ , and δ values are given out as Alpha, Beta, Gamma, and Delta, respectively.

Syntax

Alpha <alphaValue>

Beta <betaValue>

Gamma <gammaValue>

Delta <deltaValue>

Example

Alpha 1

Beta 5

Gamma 5

Delta 10

Die size and input output ports: DieSize describes the dimension of the die, namely the placement area of the design. NumInput describes the number of input pins of the die. Each "Input" syntax describes the input pin name and its location. NumOutput describes the number of output pins of the die. Each "Output" syntax describes the output pin name and its location. [Contestants cannot extend the cell size \(or region\) to solve the density violation or Flip-Flop overlap issues. The goal is to optimize the multiple objectives: timing and power; without increasing the cell's region or area \(die size\).](#)

Syntax

DieSize <lowerLeftX> <lowerLeftY> <upperRightX> <upperRightY>

NumInput <inputCount>

Input <inputName> <x-coordinate> <y-coordinate>

NumOutput <outputCount>

Output <outputName> <x-coordinate> <y-coordinate>

Example

DieSize 0 0 500 450

NumInput 2

Input INPUT0 0 25

Input INPUT1 0 5

NumOutput 2

Output OUTPUT0 500 25

Output OUTPUT1 500 5

Cell library:

Syntax

FlipFlop <bits> <flipFlopLibCellName> <libCellWidth> <libCellHeight> <pinCount>

Pin <pinName> <pinLocationX> <pinLocationY>

Example

FlipFlop 1 FF1 5 10 2

Pin D 0 8

Pin Q 5 8

FlipFlop 2 FF2 8 10 4

Pin D0 0 9

Pin D1 0 6

Pin Q0 8 9

Pin Q1 8 6

FlipFlop 2 FF2A 10 10 4

Pin D0 0 9

Pin D1 0 6

Pin Q0 8 9

Pin Q1 8 6

Placement result: The x and y coordinate describes the bottom-left corner of the cell:

Syntax

NumInstances <instanceCount>

Inst <instName> <libCellName> <x-coordinate> <y-coordinate>

Example:

NumInstances 2

Inst C1 FF1 50 20

Inst C2 FF1 50 0

Netlist:

Syntax

NumNets <netCount>

Net <netName> <numPins>

Pin <instName>/<libPinName>

Example

NumNets 4

Net N1 2

Pin INPUT0

Pin C1/D

Net N2 2

Pin INPUT1

Pin C2/D

Net N3 2

Pin C1/Q

Pin OUTPUT0

Net N4 2

Pin C2/Q

Pin OUTPUT1

Max placement utilization ratio:

Syntax

BinWidth <width>

BinHeight <height>

BinMaxUtil <util>

Example

BinWidth 100

BinHeight 100

BinMaxUtil 75

The formulation of utilization ratio of a Bin = $\frac{\sum(\text{area of each cell on the bin})}{\text{area of the bin}}$

Placement rows. The given rows would start from (0, 0) and cover the entire die:

Syntax

PlacementRows <startX> <startY> <rowWidth> <rowHeight>

Example

PlacementRows 0 0 2 10

Timing slack and delay information. For each instance pin in the design we will give out a timing slack information. The delay model is formulated by displacement delay and Q-pin delay. The definition of displacement is the Manhattan distance between the original pin location and the new pin location. For any cell displacement we times the coefficient with the displacement distance to get the displacement delay. For every flip-flop gate defined in the library we define a Q-pin delay for it.

Syntax

```
DisplacementDelay <coefficient>
QpinDelay <libCellName> <delay>
TimingSlack <instanceCellName> <PinName> <slack>
```

Example

```
DisplacementDelay 0.01
QpinDelay FF1 1
QpinDelay FF2 3
QpinDelay FF2A 2
TimingSlack C1 D 1
TimingSlack C1 Q 0
TimingSlack C2 D 1
TimingSlack C2 Q 0
```

Power consumption information: for every cell gate there is a power consumption rate.

Syntax

```
GatePower <libCellName> <powerConsumption>
```

Example

```
GatePower FF1 10
GatePower FF2 17
GatePower FF2A 18
```

3.2 Format of Output Data

The expected output is a list of bottom-left coordinates of each cell instance in the design. Contestants are expected to utilize multibit flip-flop banking and debanking techniques, therefore, the cell instance count could change, and the expected output should also describe the net connectivity information, and all net connections remain functionally equivalent as the data input with no open or short nets.

Syntax

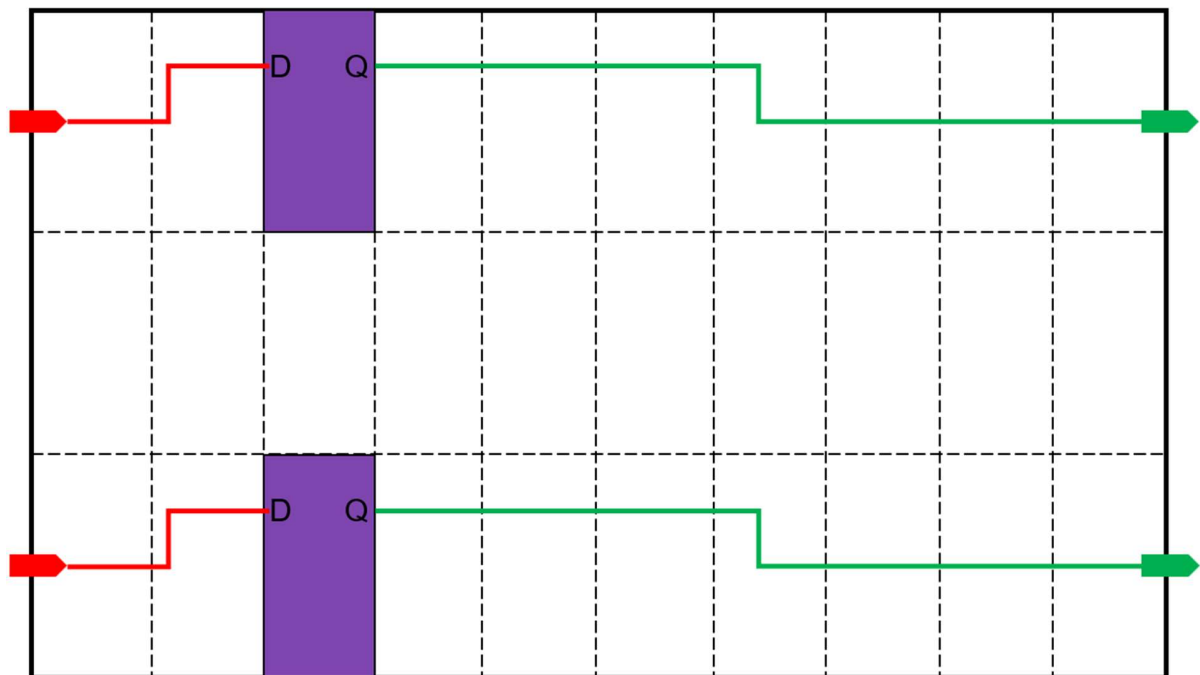
CellInst <InstCount>
Inst <instName> <locationX> <locationY> <orientation>
NumNets <netCount>
Net <netName> <numPins>
Pin <instName>/<libPinName>

Example

CellInst 1
Inst C1 FF2 48 10
NumNets 4
Net N1 2
Pin INPUT0
Pin C1/D0
Net N2 2
Pin INPUT1
Pin C1/D1
Net N3 2
Pin C1/Q0
Pin OUTPUT0
Net N4 2
Pin C1/Q1
Pin OUTPUT1

4. Example

We take the following circuit as a sample input:



	1-bit FF	2-bit FF
Normalized Area	1	1.6
Normalized Power	1	1.7
Normalized Q-Pin Delay	1	2

Alpha 1

Beta 5

Gamma 5

Delta 1

DieSize 0 0 50 30

NumInput 2

Input INPUT0 0 25

Input INPUT1 0 5

NumOutput 2

Output OUTPUT0 50 25

Output OUTPUT1 50 5

FlipFlop 1 FF1 5 10 2
Pin D 0 8
Pin Q 5 8
FlipFlop 2 FF2 8 10 4
Pin D0 0 9
Pin D1 0 6
Pin Q0 8 9
Pin Q1 8 6
NumInstances 2
Inst C1 FF1 15 20
Inst C2 FF1 15 0
NumNets 4
Net N1 2
Pin INPUT0
Pin C1/D
Net N2 2
Pin INPUT1
Pin C2/D
Net N3 2
Pin C1/Q
Pin OUTPUT0
Net N4 2
Pin C2/Q
Pin OUTPUT1
BinWidth 10
BinHeight 10
BinMaxUtil 79
PlacementRows 0 0 2 10
DisplacementDelay 0.01

QpinDelay FF1 1

QpinDelay FF2 2

TimingSlack C1 D 1

TimingSlack C1 Q 0

TimingSlack C2 D 1

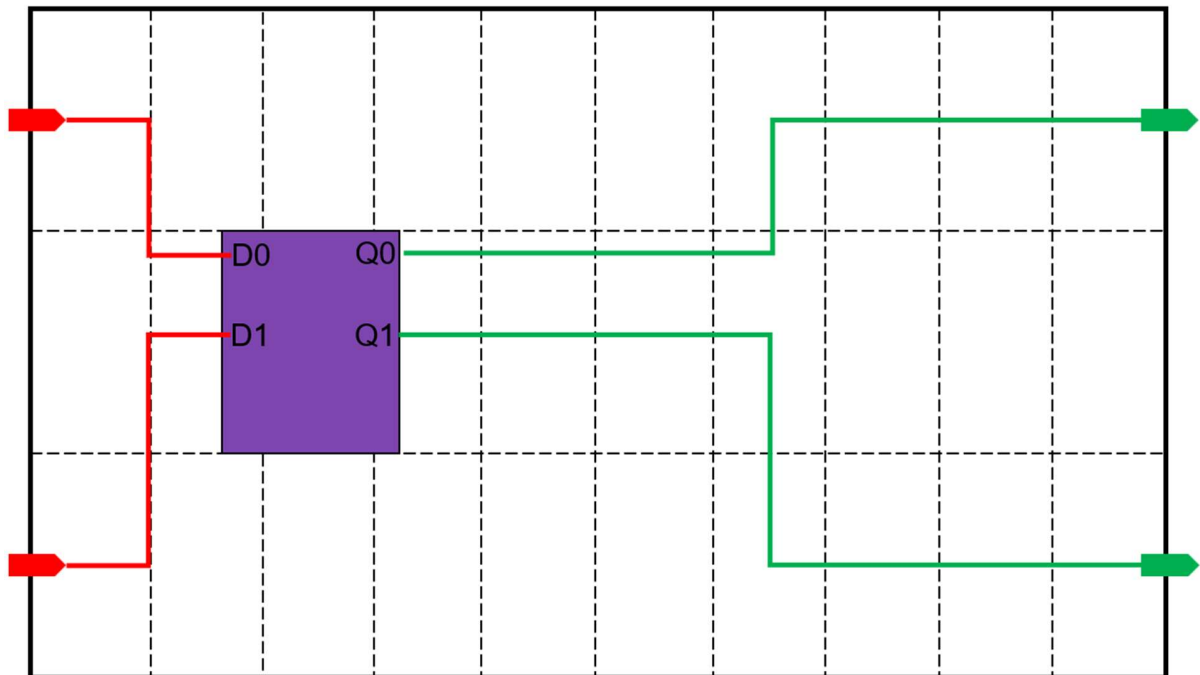
TimingSlack C2 Q 0

GatePower FF1 10

GatePower FF2 17

The simple input

Based on the given input, here's an example of output:



CellInst 1

Inst C1 FF2 8 10

NumNets 4

Net N1 2

Pin INPUT0

Pin C1/D0

Net N2 2
 Pin INPUT1
 Pin C1/D1
 Net N3 2
 Pin C1/Q0
 Pin OUTPUT0
 Net N4 2
 Pin C1/Q1
 Pin OUTPUT1

5. Evaluation

The expected output should satisfy the following constraints:

- A. All instances must be placed within the die region.
- B. All instances must be without overlap and placed on the rows
- C. Nets connected to the flip-flops must remain functionally equivalent to the data input. The result should not leave any open or short net.

Initial score is calculated by taking weighted timing, power, area, and utilization rate constraints into consideration. The cost metrics is as follows:

$$Initial\ score = \sum_{\forall i \in FF} (\alpha \cdot TNS(i) + \beta \cdot Power(i) + \gamma \cdot Area(i)) + \delta \cdot D$$

If the program and the output data violate any of these above bullets, you will get 0 score for the corresponding test case.

4.1 Runtime factor

Runtime limit is 60 minutes for each case in the evaluation machine. The hidden cases will be in the same scale as public cases. We would like to introduce the runtime factor in this contest as formulated below to encourage the ideas with faster turnaround-time.

The number of CPU cores available for your program is 8 cores in the evaluation:

$$runtime\ factor = 0.02 \times \log_2 \left(\frac{elapsed\ time\ of\ test\ binary}{median\ elapse\ time} \right)$$

$$runtime\ factor\ bounded = \max(-0.1, \min(0.1, runtime\ factor))$$

$$Final\ score = Initial\ score \times (1.0 + runtime\ factor\ bounded)$$

With this proposed runtime factor, that means if the binary is 2x faster/slower, it would get 2% of initial score advantage/disadvantage. If the binary is 4x faster/slower, it would get 4% of initial score advantage/disadvantage. The runtime factor is bounded to 10%.

4.2 Program Requirements

Your program should be able execute like following:

```
./$binary_name <input.txt> <output.txt>
```

The rule for binary name will be given out in the contest website. Please note that contestants should follow the naming description otherwise the score will be annulled.

Contestants will have access to the machine provided by ICCAD. Contestants are required to upload their binary along with every modules required to run their binary. Contestants should note that if the public account could not run the contestant's binary due to any reason, the score would result in failure.

5. References

- [1] Ya-Chu Chang, Tung-Wei Lin, Iris Hui-Ru Jiang, and Gi-Joon Nam. 2019. Graceful Register Clustering by Effective Mean Shift Algorithm for Power and Timing Balancing. In Proceedings of the 2019 International Symposium on Physical Design (ISPD '19).
- [2] Meng-Yun Liu, Yu-Cheng Lai, Wai-Kei Mak, and Ting-Chi Wang. 2022. Generation of Mixed-Driving Multi-Bit Flip-Flops for Power Optimization. In Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD '22).
- [3] Gang Wu, Yue Xu, Dean Wu, Manoj Ragupathy, Yu-yen Mo, and Chris Chu. 2016. Flip-flop clustering by weighted K-means algorithm. In Proceedings of the 53rd Annual Design Automation Conference (DAC '16)